

Enterprise RAG Platform

A multi-tenant document-intelligence system that answers natural-language questions **strictly from your own documents** — hybrid retrieval, cross-encoder reranking, grounded refusal, and real-time streaming from a **fully local LLM**. No GPU farm, no per-token API bill, no data leaving the network.

Python 3.12 · FastAPI

PostgreSQL · pgvector + FTS

Ollama · local LLM

GLM-OCR vision

Hybrid retrieval

Multi-tenant

MIT License

"A better answer starts with better evidence."

1 What is this?

Most “chat with your PDF” demos fall apart in a real organization: they can’t tell which policy is authoritative, they leak one team’s documents into another team’s answers, and they cheerfully hallucinate when the answer isn’t in the source material.

Enterprise RAG Platform is built for that reality. It ingests documents (including scanned PDFs, tables, and charts via a vision-OCR pipeline), stores **project-** and **company-scoped** knowledge, retrieves evidence through **dense + sparse hybrid search**, fuses rankings with **RRF**, sharpens precision with a **cross-encoder**, trims redundancy with **MMR**, refuses to answer when evidence is weak, and streams grounded answers from a **locally hosted LLM** — with tenant isolation enforced in SQL, not left to the model’s goodwill.

The core idea: the retrieval pipeline does the heavy lifting. Six ranking stages hand the model near-perfect context, so it produces sharp, grounded answers **even from a small ~2B-class model running locally**.

2 At a glance — key features

◆ Multi-tenant knowledge spaces

Project-private documents plus company-wide shared policies, isolated at the database layer.

◆ Hybrid retrieval

Dense semantic search and PostgreSQL full-text search, run in parallel per scope.

◆ Rank fusion + reranking

RRF merges the two lists; a cross-encoder re-reads question + passage together for precision.

◆ MMR diversification

Removes near-duplicate context so the model sees six complementary angles, not one restated six times.

◆ Grounded refusal

Below a relevance threshold, the system declines instead of hallucinating.

◆ Vision-OCR ingestion

Per-page routing: text layer, PyMuPDF `find_tables`, GLM-OCR vision for scans/charts, Tesseract fallback.

◆ Background worker

Non-blocking uploads with observable `queued` → `running` → `done` (or `failed`) jobs.

◆ Embeddable chat widget

One `<script>` tag, Shadow-DOM isolated, white-labeled, with voice input and live table/chart rendering.

◆ Tabbed admin console

Projects, a live API monitor, a domain whitelist, and global knowledge-base / cross-linking toggles.

◆ Fully local & streaming

Embeddings and generation run on Ollama; OpenAI-compatible `/v1/chat/completions` over SSE.

3 The retrieval funnel — the layers

Every question passes through a funnel that starts **wide** (don't lose the right evidence) and ends **narrow** (hand the model only what matters). Both searches run per scope — this project + the shared company scope — in parallel. This staged design is what lets a small local model punch far above its weight.

1 Dense search — meaning · cosine · top 20

Catches paraphrases the keywords miss — *"time off"* ↔ *"annual leave."*

2 Sparse search — keywords · Postgres FTS · top 20

Catches exact terms embeddings miss — policy codes, acronyms, IDs, names.

3 RRF fusion — $\Sigma 1 / (60 + \text{rank})$ · ~40 candidates

Merges the two rankings without forcing incompatible scores onto one scale.

4 Cross-encoder rerank — + project-scope boost (×1.05)

Reads question + chunk jointly for precision — run only on the shortlist, never the full corpus.

5 MMR diversify — $\lambda = 0.7$ · final 6 chunks

Kills near-duplicates so the six chunks cover six angles, not one restated six times.

6 Grounded-refusal gate — below threshold → decline

Weak evidence yields a controlled *"I don't have that"* instead of a hallucination.

Two-stage by design: the expensive cross-encoder never touches the full corpus — only the ~40 candidates that survive fusion. Broad recall stays cheap; final precision stays affordable.

4 How it works — two journeys

Nothing useful happens at chat time unless a document was prepared first. Two journeys meet inside PostgreSQL.

Journey A — Ingestion (once per document)

upload → pre-scan → parse → chunk → embed → store

1. **Upload & pre-scan.** Admin uploads a PDF; a millisecond PyMuPDF scan classifies each page *digital* / *scanned* / *rich* and sets its scope.
2. **Parse.** Per-page routing: text layer, `find_tables` → markdown, GLM-OCR vision for scans/charts, Tesseract fallback.
3. **Chunk.** Sentence-aware, ~600 tokens with ~90-token overlap; tables & figures kept whole as atomic chunks.
4. **Embed.** Each chunk → 768-dim vector via `nomic-embed-text` (local), batched with retry.
5. **Store.** Rows carry full text, vector, `tsv`, and `project_id` + `scope` isolation tags.

Journey B — Chat (every question)

embed → search → fuse → rerank → MMR → gate → stream

1. **Receive.** Widget POSTs to `/api/<slug>/v1/chat/completions`; endpoint resolves the tenant and keeps bounded history.
2. **Embed.** The question uses the same model as ingestion — question and chunks share one vector space.
3. **Retrieve.** Dense + sparse search, filtered *in SQL* to project + company scope, then RRF → rerank → MMR (the six layers).
4. **Gate.** If the best chunk is too weak, no context is passed — the bot says it doesn't know.
5. **Generate.** A strict system prompt + context + history stream from `llama3.2` token-by-token over SSE.

5 Multi-tenant isolation

One deployment serves multiple applications, each with private documents, alongside company-wide shared policies. A **Worklog** query may retrieve *Worklog documents* + *company-wide documents* — but **never** another tenant's private documents. The boundary is enforced inside the database query, before ranking begins:

```
(scope = 'project' AND project_id = :current_project)
OR (scope = 'company')
```

The LLM is never treated as a security boundary. Disallowed rows are filtered out in SQL, so another tenant's context can't even enter the prompt — prompt injection cannot widen scope.

6 Technology stack

Layer	Technology
Backend	Python 3.12 , FastAPI — async endpoints, threadpool offloading for CPU-heavy retrieval
Database	PostgreSQL — pgvector (dense) + Full-Text Search / tsvector (sparse); SQLAlchemy ORM, Alembic migrations
PDF / vision	PyMuPDF (text + find_tables), GLM-OCR vision via Ollama, Tesseract fallback (pdf2image + Poppler)
Chunking	NLTK sentence segmentation + tiktoken token budgeting, with overlap
Embeddings	nomic-embed-text via Ollama — local, 768-dimensional
Retrieval	Reciprocal Rank Fusion · cross-encoder ms-marco-MiniLM-L-6-v2 (sentence-transformers + PyTorch) · MMR ($\lambda = 0.7$)
Generation	Local LLM via Ollama (llama3.2) — temperature 0.3, capped output, warm via keep_alive
Delivery	httpx async streaming · tenacity retry/backoff · Server-Sent Events
Frontend	HTML / CSS / vanilla JS — Shadow-DOM chat widget with XSS-safe Markdown rendering

7 System requirements

The LLM, the embedding model, and the PyTorch cross-encoder drive the hardware needs — not FastAPI/Postgres.

Component	Minimum (dev / demo)	Recommended (self-host)
CPU	4-core x86-64	8-core / 16-thread
RAM	8 GB (16 GB preferred)	32 GB
GPU	None (CPU works, slower)	NVIDIA 8 GB+ VRAM, CUDA
Disk	15 GB SSD	50 GB+ NVMe SSD
OS	Windows 10/11, macOS 12+, Ubuntu 20.04+	Windows 11, macOS 13+, Ubuntu 22.04+

First run needs internet to download models & packages; afterwards it runs fully offline.

8 Why the answers are good & safe

Two searches, not one

Meaning + keywords catch more of the right material than either alone.

Re-ranking reads jointly

Question and chunk are scored together, so final context is genuinely on-point.

Diversity over repetition

MMR keeps context varied instead of six paraphrases of one sentence.

Refuse, don't invent

Grounded threshold means "I don't know → say so," never "→ hallucinate."

Isolation in SQL

Disallowed documents never reach the prompt; the model isn't trusted as a boundary.

Local & escaped

Data never leaves the network; model output is escaped before it touches the DOM.